

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.22;

import {OFT} from "@layerzerolabs/lz-evm-oapp-v2/contracts/oft/OFT.sol";
import {ERC20Votes} from
"@openzeppelin/contracts/token/ERC20/extensions/ERC20Votes.sol";
import {ERC20Permit} from
"@openzeppelin/contracts/token/ERC20/extensions/ERC20Permit.sol";
import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import {Nonces} from "@openzeppelin/contracts/utils/Nonces.sol";
import {Ownable} from "@openzeppelin/contracts/access/Ownable.sol";

/**
 * @title OsniasClearing – Osnias Clearing Governance (SEI MAINNET – v1)
 *
 * @notice Jeton de gouvernance et de valorisation de l'écosystème Osnias
Clearing.
 *
 * Caractéristiques principales :
 * - Supply fixe : 100 000 000 OSNIAS créés à la genèse – aucune émission
ultérieure.
 * - Omnichaîne : LayerZero V2 OFT – fongible entre chaînes EVM autorisées.
 * - Gouvernance : ERC20Votes – vote exclusivement sur Polygon.
 * - Permit : ERC20Permit / ERC-2612 – signatures off-chain.
 * - Burn : volontaire, msg.sender uniquement.
 * - Confiscation: interdite – aucune fonction de burn sur les jetons d'un
tiers.
 *
 * Séparation ORUSD / OSNIAS :
 * ORUSD = instrument de clearing P2P – circuit fermé, non spéculatif.
 * OSNIAS = actif économique externe – librement transférable, coté sur AMM.
 *
 * VERSION TESTNET – v1
 */
contract OSNIAS is OFT, ERC20Permit, ERC20Votes {

    // _____
    // IDENTITÉ DU PROTOCOLE
    // _____

    string public constant PROTOCOL = "Osnias Clearing";
    string public constant PROTOCOL_VERSION = "1.0";
    string public constant TOKEN_FULL_NAME = "Osnias Clearing Governance";

    // _____
    // CONSTANTES
    // _____

    /// @notice Supply initiale maximale – non inflationniste.
    /// Aucune émission discrétionnaire après la genèse.
    /// Les seuls mints ultérieurs sont les crédits techniques OFT
    /// correspondant à des OSNIAS préalablement débités sur une autre
chaîne.
    /// Le burn volontaire peut réduire cette supply définitivement.

```

```

uint256 public constant GENESIS_SUPPLY = 100_000_000 * 10 ** 6;

/// @notice Décimales – alignées sur ORUSD (6 décimales).
uint8 public constant OSNIAS_DECIMALS = 6;

/// @notice Adresse du gestionnaire initial (Polygon Testnet).
// _____
// CONSTRUCTOR
// _____

/**
 * @notice Déploiement OSNIAS sur Sei.
 * @param _lzEndpoint Adresse de l'endpoint LayerZero sur cette chaîne.
 * @param _initialManager Adresse du gestionnaire initial (wallet ou Safe
multisig).
 *
 * @dev La totalité de la supply initiale est créée à la genèse et remise
 *       au gestionnaire initial. Aucune émission discrétionnaire n'est
 *       possible après le constructeur. Les mints OFT ultérieurs
correspondent
 *       exclusivement à des OSNIAS préalablement brûlés sur une autre
chaîne.
 */
constructor(address _lzEndpoint, address _initialManager)
    OFT("Osnias Governance Token", "OSNIAS", _lzEndpoint, _initialManager)
    ERC20Permit("Osnias Governance Token")
    Ownable(_initialManager)
{
    require(_initialManager != address(0), "OSNIAS: gestionnaire invalide");
    _mint(_initialManager, GENESIS_SUPPLY);
}

// _____
// DÉCIMALES
// _____

/**
 * @notice Retourne 6 décimales – identique à ORUSD et USDC.
 */
function decimals()
    public
    pure
    override(ERC20)
    returns (uint8)
{
    return OSNIAS_DECIMALS;
}

// _____
// BURN VOLONTAIRE
// _____

/**
 * @notice Burn volontaire – agit uniquement sur les jetons de msg.sender.

```

```

    * @dev Aucune fonction ne permet de brûler les jetons d'un tiers.
    */
function burn(uint256 amount) external {
    _burn(msg.sender, amount);
}

// -----
// OVERRIDES REQUIS PAR SOLIDITY
// -----

function _update(address from, address to, uint256 value)
    internal
    override(ERC20, ERC20Votes)
{
    super._update(from, to, value);
}

function nonces(address owner)
    public
    view
    override(ERC20Permit, Nonces)
    returns (uint256)
{
    return super.nonces(owner);
}
}

```